

NBchat 模块答辩报告

我负责的三个模块

我在本次项目中承担了**实时通信系统**、**AI 智能助手**和**前端动效系统**三个核心模块的设计与开发。下面我将从技术实现、架构设计和细节打磨三个维度，逐一介绍每个模块。

模块一：实时通信系统

实时通信是聊天应用的**心脏**——没有它，聊天就退化成了“手动刷新浏览器”。

1.1 WebSocket 消息推送

连接建立

浏览器端通过 `chat.js` 的 `connectChatSocket()` 建立 WebSocket 连接。这里有三个精心设计的细节：

自动协议协商：`const protocol = location.protocol === "https:" ? "wss" : "ws"`。如果部署了 HTTPS，WebSocket 自动升级为 WSS，否则使用 WS。这让开发环境和生产环境之间零配置切换。

HTTP Session 桥接：WebSocket 协议本身没有 Session 概念。我通过 `HttpSessionConfigurator` 在 WebSocket 握手阶段拦截 HTTP 请求，把已登录用户的 HTTP Session 注入到 WebSocket Session 的 `UserProperties` 中。这样 WebSocket 端点就能通过 `socket.getUserProperties().get(SessionKeys.USER_ID)` 获取当前用户身份，实现了 REST 和 WebSocket 的身份统一。

连接守卫：`@OnOpen` 方法首先检查 HTTP Session 是否存在、是否已登录。未登录者直接以 `VIOLATED_POLICY` 关闭连接，防止匿名用户占用 WebSocket 资源。

事件路由

`ChatEndpoint.onMessage()` 是一个轻量级的事件路由器。它先把消息解析为 JSON 树，检查 `event` 字段：

- `REACTION` → 表情反应处理 → 聚合更新后广播给全会话
- `RECALL` → 消息撤回处理 → 2 分钟时间窗口校验 → 广播撤回更新
- 其他 → 普通消息 → 写入数据库 → 触发 AI 检测 → 广播

每种事件的处理结果都以 JSON 形式通过 `SocketRegistry` 广播。这个设计让一种 WebSocket 连接承载了三种完全不同的业务逻辑，避免了为每个功能开一个独立连接。

SocketRegistry 底层设计

最初的实现中，`ChatEndpoint` 和 `VoiceEndpoint` 各自维护一个独立的 `ConcurrentHashMap<Long, Set<Session>>`。这有两个问题：用户在线状态在两个 channel 间不可见；如果未来增加第三个 Endpoint（例如通知推送），又要复制一套代码。

重构后的 `SocketRegistry` 使用单例模式，内部用双层 Map 架构：`ConcurrentHashMap<String, ConcurrentHashMap<Long, Set<Session>>>`。外层 key 是 channel 名（"chat" / "voice"），内层 key 是用户 ID。`shared()` 工厂方法返回全局唯一实例。`send()` 方法遍历一个用户的所有活跃 Session 逐一推送，单个连接的 `IOException` 被静默吞掉——不会因为一个僵尸连接阻塞其他连接的投递。

错误可见性

消息发送失败时，服务器不会默默丢弃错误。`ChatEndpoint` 的 try-catch 块捕获所有异常，构造一个 `{"event": "ERROR", "message": "..."}` 的 JSON 响应，只发送给发起者本人。前端收到后在消息列表中渲染一个带红色感叹号的失败标记。这样用户能立即知道自己发的消息为什么没成功（例如"对方已不在你的好友列表中"）。

离线降级

当 WebSocket 不可用时（例如网络波动、浏览器不支持），`sendMessage()` 自动降级为 REST API 调用 `POST /chat/messages`。如果 REST 也失败，则渲染发送失败的红色标记。三层降级：WebSocket → REST → 错误展示。

1.2 语音通话信令系统

语音通话采用 **WebRTC 媒体流 + WebSocket 信令** 的双通道架构。

信令流程

这是一段精心编排的六步握手：

1. **发起呼叫**：前端调用 `POST /voice/calls/{calleeId}` → `VoiceService.start()` 通过 `BusyUserRegistry.reserve()` 原子性标记双方为"忙线"，然后写入 `voice_call_sessions` 表，状态为 `RINGING`
2. **推送邀请**：通过 Voice WebSocket 向被叫方发送 `call-invite` 信号
3. **来电弹窗**：被叫方前端弹出 `<dialog>` 模态框，显示"收到语音通话"和接听/拒绝按钮
4. **接受呼叫**：被叫点击接听 → `POST /calls/{id}/accept` → 状态更新为 `ACCEPTED` → 通过信令信道回复 `call-accepted`
5. **WebRTC 协商**：双方各自创建 `RTCPeerConnection`，交换 offer/answer/ICE candidate。这些信令数据全部通过 Voice WebSocket 中转，不经过服务器存储
6. **媒体流建立**：`ontrack` 事件触发，远端音频流被挂载到隐藏的 `<audio>` 元素，通话开始

BusyUserRegistry 忙线管理

用 `ConcurrentHashMap.newKeySet()` 维护一个线程安全的忙线用户集合。`reserve(callerId, calleeId)` 方法用原子操作检查双方是否空闲，如果一方正在通话中则拒绝新呼叫。通话结束（ENDED/REJECTED/MISSED）时自动释放标记。这个设计保证了同一用户不可能同时参与两通电话。

ICE 服务器配置

`IceServerConfig.from()` 从 `app.properties` 读取 STUN 和 TURN 服务器配置。开发环境只需 Google 公共 STUN，生产环境可以配置自建 coturn 服务来穿透对称 NAT。配置方式通过环境变量注入，不在代码中硬编码。

模块二：AI 智能助手

2.1 从本地 Ollama 到云端 DeepSeek

AI 助手模块经历了关键技术选型变更。最初的方案是在服务器上部署本地 Ollama 模型 (qwen2.5:7b)，但 7B 模型需要 4-5GB 内存，我们的 2GB 云服务器完全无法承载。经过技术评估后，切换为 DeepSeek 云端 API。

DeepSeek 的优势：

- API 格式与 OpenAI 兼容，生态成熟
- 中文理解能力强，适合群聊场景
- 定价极低（约 1 元/百万 token），适合学生项目
- 不需要服务器端 GPU 或大内存

2.2 DeepSeekClient 实现

DeepSeekClient 是 AI 助手的通信层，核心设计要点：

零外部依赖：使用 Java 11 内置的 `java.net.http.HttpClient`，不需要引入 OkHttp 或 Retrofit，WAR 包体积不受影响。

Bearer Token 鉴权：通过 `Authorization: Bearer {apiKey}` 头传递 API Key。API Key 从 `AppConfig.get("deepseek.apiKey")` 读取，支持环境变量覆盖——生产环境通过 `DEEPSEEK_APIKEY` 环境变量注入，不会写死在配置文件中。

Transport 接口抽象：定义了 `Transport` 函数式接口，默认实现是 `HttpClientTransport`。在单元测试中可以通过注入 Mock Transport 来模拟 API 响应，实现完全离线测试。这是可测试性设计的关键。

超时与容错：45 秒超时比 Ollama 本地模型的 30 秒更宽松。任何网络异常（`IOException` 或 `InterruptedException`）都被捕获，返回用户友好的提示信息而非抛出异常——这样 AI 失败不会影响正常消息发送。

忽略未知字段：`@JsonIgnoreProperties(ignoreUnknown = true)` 注解意味着 DeepSeek API 新增字段时不会导致反序列化崩溃，保证向后兼容。

2.3 AiAssistantService 提示词工程

AiAssistantService 负责构造发送给 AI 的完整提示词。它的工作流程分四步：

1. **检测 @提及：**`isMentioned()` 判断消息内容是否包含 @千问小助手。只有在群聊中检测到 @才触发 AI，避免误调用。
2. **提取用户问题：**`extractUserPrompt()` 从 @千问小助手 后面提取实际的问题文本。如果用户只发了 @千问小助手 没有具体问题，则使用默认提示词“请根据最近聊天进行总结”。
3. **构建上下文：**`buildPrompt()` 取最近 30 条聊天记录作为对话上下文。每条记录格式化为 `{发送者}: {内容}`，让 AI 理解群聊中的对话流。
4. **调用 API：**将 system prompt（定义 AI 角色和行为规范）和 user prompt（问题 + 上下文）发送给 DeepSeek。

System prompt 的设计：“你是群聊中的中文智能助手，名字是千问小助手。请基于群聊上下文回答用户问题，回答要准确、简洁、可执行。”这四句话定义了 AI 的身份、能力边界和输出风格。

2.4 前端交互

发送 @千问小助手 消息时，前端立即在消息列表中插入一条"千问小助手正在分析群聊内容..."的 typing 动画（CSS `typing-dots` 三个跳动圆点）。1.8 秒后自动重新加载历史消息，此时 AI 回复已经在数据库中并出现在聊天流中。如果 Ollama/DeepSeek 返回错误（模型不可用、网络超时等），AI 消息的内容是友好的错误提示，不会影响正常聊天。

模块三：前端动效系统

所有动效均使用原生 CSS 和 JavaScript 实现，零第三方动画库，保持了 WAR 包的轻盈。

3.1 Canvas 粒子引擎

这是整个动效系统的核心技术。`triggerSendParticles()` 函数在用户点击发送按钮时触发。

粒子生成：从发送按钮的中心位置，以扇形角度（ $-\pi/2 \pm 1.2$ 弧度）随机发射 34 个圆形粒子。每个粒子有随机速度（2.5~8.0）、随机大小（2~5.5px）、随机生命值（28~46 帧）。颜色从 5 色调色板中循环选取。

物理模拟：`animateFxParticles()` 是 `requestAnimationFrame` 驱动的渲染循环。每帧更新粒子位置（`x += vx`、`y += vy`）、施加重力（`vy += 0.12`）、递减生命值。透明度按生命值比例衰减（`globalAlpha = life / maxLife`），实现渐隐效果。当所有粒子耗尽后自动停止动画循环并清除 Canvas。

HiDPI 适配：Canvas 的实际像素尺寸乘以 `devicePixelRatio`，然后通过 CSS 缩放回逻辑尺寸，保证在 Retina 屏幕上不模糊。同时用 `setTransform(ratio, 0, 0, ratio, 0, 0)` 缩放坐标系统，使得所有绘图代码仍以 CSS 像素为单位。

无障碍适配：检测 `prefers-reduced-motion: reduce` 媒体查询，当用户系统设置了"减少动效"时完全跳过粒子生成。

3.2 消息气泡弹簧入场

`@keyframes bubbleSpringIn` 定义了消息气泡的弹性入场动画。`cubic-bezier(.34, 1.56, .64, 1)` 的缓动曲线超出了标准范围（贝塞尔控制点 y 值 > 1），产生了"超出→回弹→归位"的弹簧效果。动画持续 460ms，保证视觉上有冲击力但不拖沓。

3.3 @提及脉冲高亮

当消息内容包含 @ 字符时，`markMentionPulse()` 给消息行添加 `mention-pulse` class。CSS `@keyframes mentionPulse` 使用 `box-shadow` 动画：外发橙色光晕从 0 → 7px → 14px → 0，持续 2.1 秒。2.2 秒后 JavaScript 自动移除 class，防止 DOM 状态残留。

3.4 图片渐进加载

`wireImageReveal()` 函数给所有带 `.chat-image.loading` 的图片元素注册 `load/error` 事件监听器。图片的 CSS 初始状态是 `filter: blur(14px) + opacity: 0.68`。加载完成后通过 `@keyframes imageReveal` 动画在 420ms 内过渡到 `blur(0) + opacity: 1`。对于已经缓存的图片，`image.complete` 为 true 时通过 `requestAnimationFrame` 立即展现，避免闪烁。

3.5 朋友圈爱心粒子

点赞按钮点击时，`spawnLikeHearts()` 生成 7 个 ♥ 字符。每个爱心使用 CSS 变量 `--heart-x` 和 `--heart-y` 控制随机偏移方向，`--heart-delay` 控制错开延迟（0~238ms）。`@keyframes heartFloat` 动画让爱心从按钮中心向上飘浮，同时旋转 14° 并渐隐。`animationend` 事件自动清理 DOM 元素。

3.6 阅后即焚刮刮乐

`wireBurnCanvases()` 函数给每个焚毁消息初始化一个 Canvas 覆盖层。Canvas 使用 `fillRect` 铺满深色背景并绘制"刮开查看"提示文字。

擦除机制：用户按住鼠标或手指在 Canvas 上滑动时，`scratch()` 函数设置 `globalCompositeOperation = "destination-out"`，然后用 `arc()` 绘制圆形——这会"挖掉"Canvas 上的像素，露出下层 DOM 中的消息文本。擦除完成后切回 `source-over`。

自动销毁：`burnRevealRatio()` 对 Canvas 的 Alpha 通道进行采样（每 40 个像素采样一次），计算透明度比例。当超过 18% 的像素被刮开时，先调用 `POST /messages/{id}/burn-read` 通知服务器标记已读，然后启动 8.5 秒倒计时。倒计时结束后添加 `burned` class 隐藏消息。

技术亮点总结

模块	核心技术
WebSocket 推送	HTTP Session 桥接、事件路由、三层降级、SocketRegistry 单例
语音信令	offer/answer/ICE 三段协商、BusyUserRegistry 原子忙线管理、STUN/TURN 可配置
AI 助手	OpenAI 兼容 API、Transport 可测试抽象、@JsonIgnoreProperties 向后兼容、提示词工程
Canvas 粒子	requestAnimationFrame 物理引擎、HiDPI 适配、prefers-reduced-motion 无障碍
CSS 动效	cubic-bezier 弹簧曲线、box-shadow 脉冲、blur 渐变加载、CSS 变量参数化动画
刮刮乐	globalCompositeOperation destination-out、Alpha 通道采样、pointer/touch 双事件支持